CSE 414 Final Project

~Developing a Database for a Producer Company~

In this project, a database for a regular company has been developed. It includes necessary transactions, views, roles, joins, and more. The following steps outline the entire database development process:

1. User Requirements

In the first step, the user requirements for the company must be thoroughly defined. For this specific producer company, the primary goal is to facilitate the sale of products that employees individually produce to customers. Given this objective, the requirements are specified in detail as follows:

- **Customer Purchase Capability**: A customer must have the ability to browse and purchase products offered by the company. This includes viewing product details, adding products to a order.
- **Order History Access**: Customers should have access to a comprehensive history of their previous orders. This includes viewing details of past purchases, such as product descriptions, quantities, prices, order dates, and delivery statuses.
- **Customer Information Management**: Customers must be able to update their personal information, including name, address, contact details, and payment information. Ensuring that customers can easily maintain accurate and up-to-date information is crucial for effective communication and service delivery.
- **Employee Production Capability**: Employees should be empowered to produce products that will be available for sale. This involves creating new product entries in the database, specifying product details such as name, price, etc. This capability ensures that the product catalog remains current and diverse.
- **Product Management by Employees**: Employees must have the authority to manage the products they produce. This includes the ability to remove their products from the catalog if necessary. Such functionality is important for maintaining the quality and relevance of the product offerings.
- **Administrative Oversight and Control**: An administrator must have comprehensive control over the entire database. This includes the ability to create, read, update, and delete any records, manage user accounts and permissions, monitor transactions, and ensure the integrity and security of the database. Admins play a critical role in maintaining the overall system health and operational efficiency.
- **Data Storage Considerations**: Given that the company is a low-scale operation, it is important to recognize that there is no need for extensive data storage solutions. The database should be optimized to handle a smaller volume of transactions and data entries efficiently. This consideration helps in reducing operational costs and complexity while ensuring the database remains responsive and easy to maintain.

These detailed requirements provide a comprehensive understanding of the necessary features and functionalities that the database must support to meet the company's operational needs effectively.

2. Entity-Relationship (E-R) Diagram

Initially, a basic E-R diagram is designed, including the necessary fields and essential relationships between tables, although it is not yet normalized. This diagram serves as a starting point for further refinement and normalization.

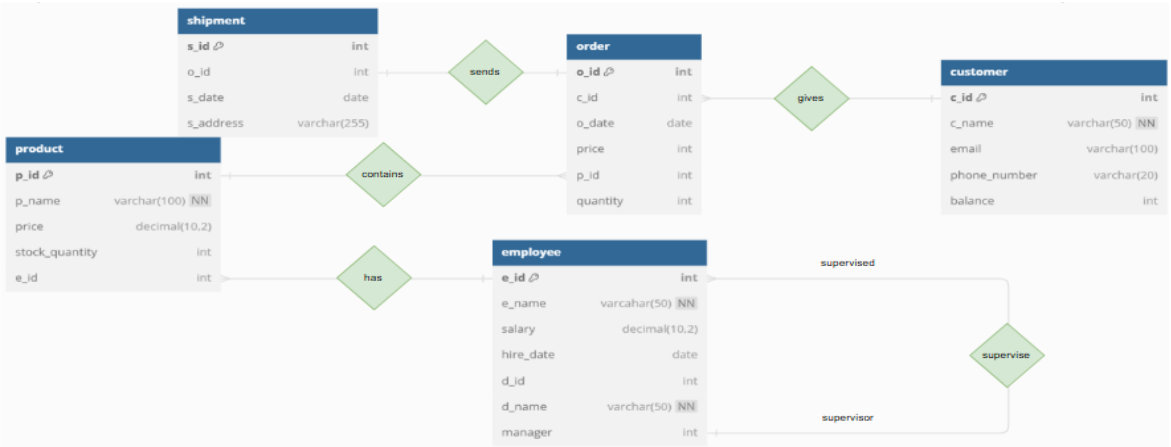


Figure 1: Initial E-R diagram for database

Initial E-R diagram have following relations:

- A customer gives many orders, but an order can belongs to only one customer (many – 1).
- An order is shipped by a shipment and every shipment can belong to one costumer (1 – 1).
- An order contains one product, but a product can be added to many orders (1 – many).
- An employee has many products, but a product can belong to one employee (many – 1).
- An employee is supervised by 1 employee, but an employee can supervise many other employees (1 – many).

3. Functional Dependencies

To normalize the database, following requirements must be met.

1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF, and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF, and no transition dependency exists.
BCNF	A relation will be in BCNF if it is in 3NF, and for functional dependency X–>Y, if X must be a candidate key.

Table 1: Normalization forms and their requirements

To check the table meets the condition for normalization forms, it is required to find functional dependencies for each table. The functional dependency $\boldsymbol{\alpha} \rightarrow \boldsymbol{\beta}$ holds on R if and

only if for any legal relations r(R), whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t1[\alpha] = t2\ [\alpha] \Rightarrow t1[\beta] = t2\ [\beta]$$

Using this formula, we can find all functional dependencies as follows:

1. Shipment:
   s_id -> o_id, s_date, s_address
2. Product
   p_id -> p_name, price, stock_quantity, e_id
3. Customer
   c_id -> c_name, phone, email, balance
4. Order
   o_id -> c_id, o_date, p_id,
   o_id, p_id -> quantity
   p_id, quantity -> price
5. Employee
   e_id -> e_name, salary, hire_date, d_id
   d_id -> d_name, manager

According to the functional dependencies, the normalization form of the database is as follows:

- There is no multivalued attribute in the database, so all tables meet the conditions for **1NF**.
- All non-key attributes are fully functional dependent on the primary key, so all tables meet the conditions for **2NF**.
- There are some tables, order and employee, which have transition dependency, so it is not **3NF**.
- All tables meet conditions for **BCNF** after normalization for 3NF.

4. Normalization

After determining the functional dependencies for all tables, any functional dependencies that prevent the database from achieving the desired normalization forms need to be addressed and removed. This process will enable the database to be fully normalized.

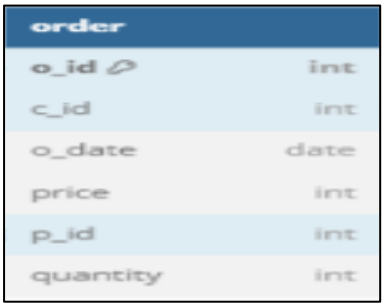


Figure 2: Before normalization

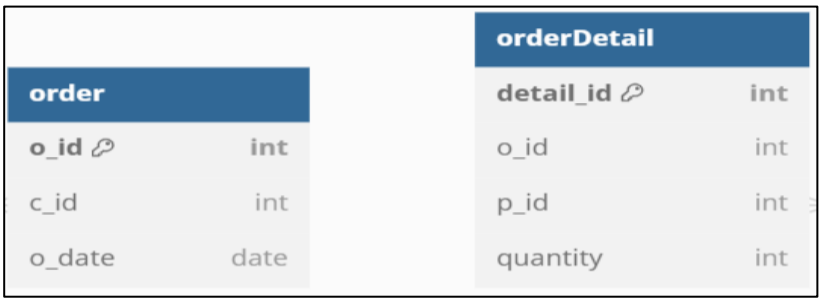


Figure 3: After normalization



Figure 4: Before normalization

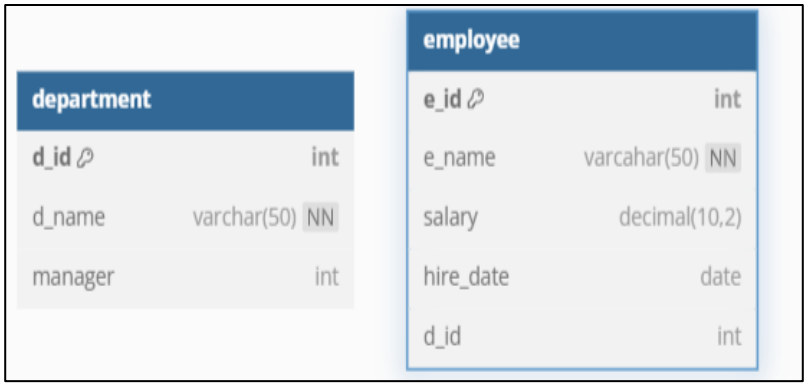


Figure 5: After normalization

After normalizing these tables, the new version of E-R diagram as follows:

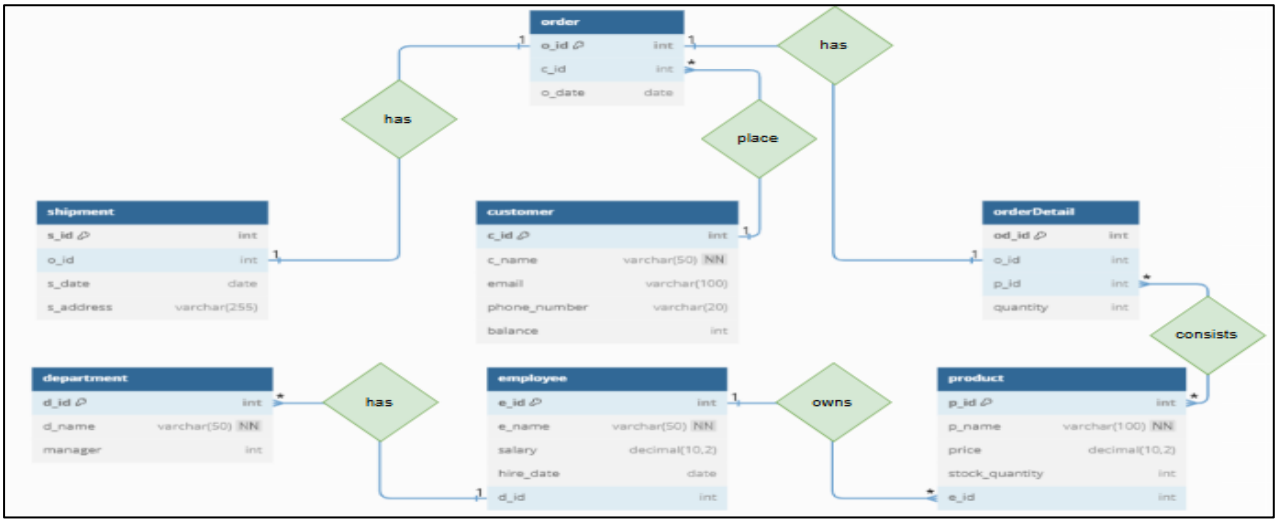


Figure 6: Final E-R Diagram

5. Database Schema

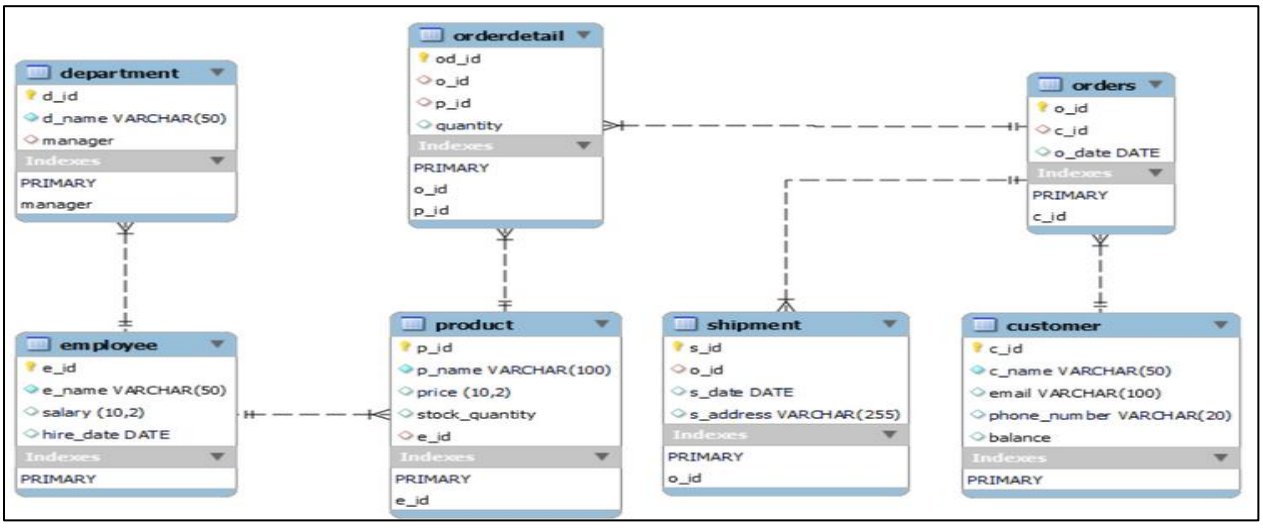


Figure 7: Database Schema

6. Database Implementation

The database is implemented using MySQL. All tables in the database schema have been created, and some data has been added to demonstrate database triggers, views, transactions, roles, and other functionalities.

7. User Interface

A simple user interface has been designed to manage the database. Users must log in to the database management system using their role and password, ensuring that their privileges are predefined and preventing potential issues. After logging in, the interface provides necessary functionalities through buttons, such as displaying tables, and performing insert, update, and delete operations. Users must have the necessary permissions to perform these operations. If a user does not have permission to perform an action, the button will respond with "access denied."

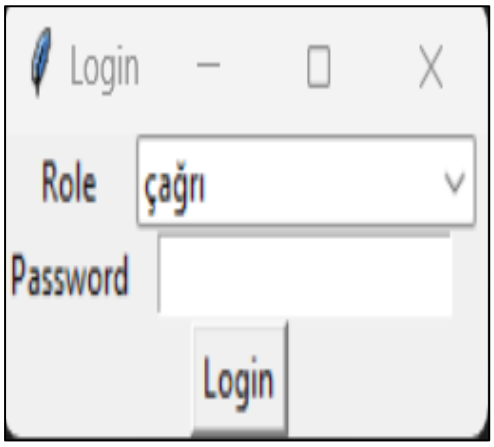


Figure 8: Login to database management system

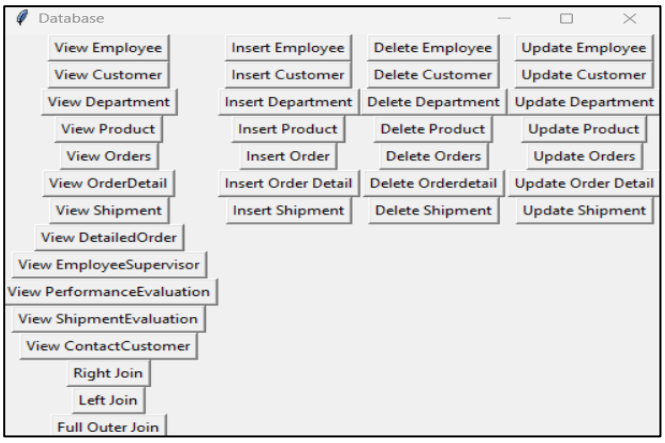
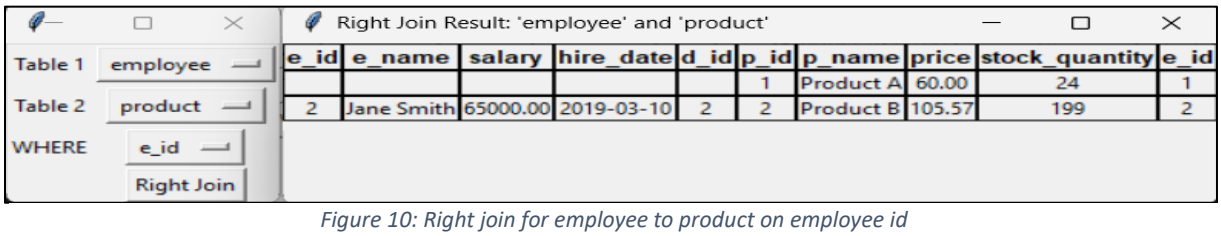


Figure 9: Database management system

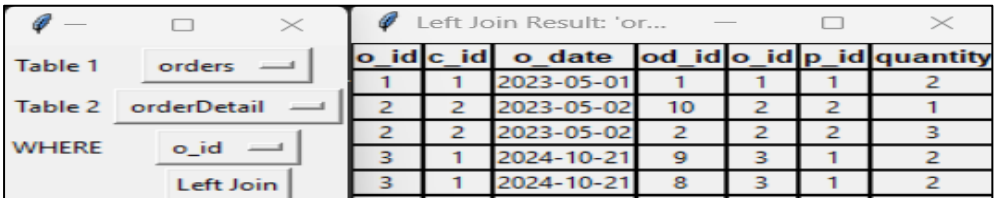
8. Query Development

Many different SQL queries are used when managing the database. Additionally, join operations have been added to the user interface.



e_id	e_name	salary	hire_date	d_id	p_id	p_name	price	stock_quantity	e_id
					1	Product A	60.00	24	1
2	Jane Smith	65000.00	2019-03-10	2	2	Product B	105.57	199	2

Figure 10: Right join for employee to product on employee id



o_id	c_id	o_date	od_id	o_id	p_id	quantity
1	1	2023-05-01	1	1	1	2
2	2	2023-05-02	10	2	2	1
2	2	2023-05-02	2	2	2	3
3	1	2024-10-21	9	3	1	2
3	1	2024-10-21	8	3	1	2

Figure 11: Left join for order to orderDetail on order id

9. Triggers

5 triggers have been designed for this project.

Name	Event	Table	Timing
addressChange	UPDATE	customer	BEFORE
updateStockQuantity	INSERT	orderdetail	BEFORE
cancelOrder	DELETE	orders	BEFORE
createProduct	INSERT	product	BEFORE
updatePrice	UPDATE	product	BEFORE

Figure 12: Triggers in database

- **addressChange**: Changes the address of the shipment when customer address is changed.
- **updateStockQuantity**: Updates the stock quantity of a product when it is added to orderDetail table.
- **cancelOrder**: Refunds the money of the order to customer.
- **createProduct**: Raises the salary of employee when it creates a product.
- **updatePrice**: Updates the price of the product according to stock quantity.

1. addressChange Trigger

s_id	o_id	c_id	s_address	address
1	1	1	123 Main St, Anytown, USA	123 Main St, Anytown, USA
2	2	2	456 Oak St, Othertown, USA	456 Oak St, Othertown, USA

Figure 13: Shipment and customer addresses before updating customer address

```
73 •  UPDATE customer SET customer.address = "Turkey" where customer.c_id = 1;
74
75 •  SELECT shipment.s_id, orders.o_id, customer.c_id, shipment.s_address, customer.address
76    from shipment, orders, customer
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

s_id	o_id	c_id	s_address	address
1	1	1	Turkey	Turkey
2	2	2	456 Oak St, Othertown, USA	456 Oak St, Othertown, USA

Figure 14: Shipment and customer addresses after updating customer address

2. cancelOrder Trigger

c_id	balance	o_id	od_id	quantity	p_id	price
1	100	1	1	2	1	10.50
2	200	2	2	3	2	20.75

Figure 15: Before cancelling order

```
79    DELETE FROM orders WHERE o_id = 1;
80 •  SELECT * from customer;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

c_id	c_name	email	phone_number	balance	address
1	Alice Brown	alice@example.com	123-456-7890	121	Turkey
2	Charlie Green	charlie@example.com	987-654-3210	200	456 Oak St, Othertown, USA
NULL	NULL	NULL	NULL	NULL	NULL

Figure 16: After cancelling order

3. updateStockQuantity & updatePrice Trigger

p_id	p_name	price	stock_quantity	e_id
1	Product A	10.50	100	1
2	Product B	20.75	200	2
NULL	NULL	NULL	NULL	NULL

Figure 17:Stock quantities and prices before inserting an order

```
87      INSERT INTO orderDetail VALUES(3, 3, 2, 100);
88 •    SELECT * FROM product;
```

Result Grid | Filter Rows: | Edit:

p_id	p_name	price	stock_quantity	e_id
1	Product A	10.50	100	1
2	Product B	41.50	100	2
NULL	NULL	NULL	NULL	NULL

Figure 18: Stock quantities and prices after inserting an order

4. createProduct Trigger

e_id	e_name	salary	hire_date
1	John Doe	60000.00	2020-01-15
2	Jane Smith	65000.00	2019-03-10
3	Bob Johnson	55000.00	2021-07-01
NULL	NULL	NULL	NULL

Figure 19: Salaries before creating a product

```
91      INSERT INTO product VALUES (4, "shoes", 1000, 50, 1);
92 •    SELECT * FROM employee;
```

Result Grid | Filter Rows: | Edit: | Export

e_id	e_name	salary	hire_date
1	John Doe	61500.00	2020-01-15
2	Jane Smith	65000.00	2019-03-10
3	Bob Johnson	55000.00	2021-07-01
NULL	NULL	NULL	NULL

Figure 20: Salaries after creating a product

10. Views

1. **contanctCustomer:** If a product has problems, gets the contact info of customer to inform him.

```
124 •   DROP VIEW IF EXISTS ContactCustomer;
125 •   CREATE VIEW ContactCustomer AS -- If a product has problems, get the contact info of customer to inform him.
126     SELECT customer.c_id, customer.c_name, phone_number, product.p_id, product.p_name
127     FROM customer, product, orders, orderDetail
128     WHERE orders.c_id = customer.c_id and orderDetail.o_id = orders.o_id and product.p_id = orderDetail.p_id;
129
130 •   SELECT * from ContactCustomer;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

c_id	c_name	phone_number	p_id	p_name
1	Alice Brown	123-456-7890	1	Product A
2	Charlie Green	987-654-3210	2	Product B

Figure 21: View for contact customer

2. **detailedOrder:** Gets the total price of an order.

```
95 •    CREATE VIEW DetailedOrder AS
96      SELECT orders.o_id, orders.c_id, orderDetail.quantity, product.price, orderDetail.quantity * product.price as total_amount
97      FROM orders, orderDetail, product
98      WHERE orders.o_id = orderDetail.o_id and orderDetail.p_id = product.p_id;
99
100 •   SELECT * from DetailedOrder;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

o_id	c_id	quantity	price	total_amount
3	2	100	41.50	4150.00
2	2	3	41.50	124.50

Figure 22: View for detailed order

3. **employeeSupervisor:** Gets the name of the employee's supervisor to complain about him.

```
102 •   DROP VIEW IF EXISTS EmployeeSupervisor;
103 •   CREATE VIEW EmployeeSupervisor AS
104     SELECT e_id, e_name, manager as supervisor
105     FROM employee, department
106     WHERE employee.d_id = department.d_id;
107
108 •   SELECT * FROM EmployeeSupervisor;
```

Result Grid | Filter Rows: | Export:

e_id	e_name	supervisor
1	John Doe	3
2	Jane Smith	2
3	Bob Johnson	1

Figure 23:View for employee supervisor

4. **PerformanceEvaluation:** Evaluates employees in terms of their number of products.

```
110 •   DROP VIEW IF EXISTS PerformanceEvaluation;
111 •   CREATE VIEW PerformanceEvaluation AS
112     SELECT e_id, COUNT(p_id) FROM product group by e_id;
113
114 •   SELECT * FROM PerformanceEvaluation;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

e_id	COUNT(p_id)
1	1
2	1

Figure 24: View for performance evaluation

5. **shipmentEvaluation:** Evaluate shipment in terms of delivery date.

```
116 •   DROP VIEW IF EXISTS ShipmentEvaluation;
117 •   CREATE VIEW ShipmentEvaluation AS
118     SELECT s_id, o_date, s_date, s_date - o_date as delivery_time
119     FROM shipment, orders
120     WHERE shipment.o_id = orders.o_id;
121
122 •   SELECT * FROM ShipmentEvaluation;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

s_id	o_date	s_date	delivery_time
1	2023-05-01	2023-05-03	2
2	2023-05-02	2023-05-04	2

Figure 25: View for shipment evaluation

11. Transactions

1. Updating balance after an order placed

c_id	c_name	email	phone_number	balance	address
1	Alice Brown	alice@example.com	123-456-7890	100	123 Main St, Anytown, USA
2	Charlie Green	charlie@example.com	987-654-3210	200	456 Oak St, Othertown, USA
NULL	NULL	NULL	NULL	NULL	NULL

Figure 26: Customers and their balances

```
START TRANSACTION;
INSERT INTO orderDetail VALUES (9, 3, 1, 2);
SET @total_amount := NULL;
SELECT quantity * price INTO @total_amount FROM orderDetail, product WHERE orderDetail.od_id = 8 and orderDetail.p_id = product.p_id;
UPDATE customer
SET balance = balance - @total_amount
WHERE customer.c_id IN (
SELECT c_id FROM orders, orderDetail
where orderDetail.od_id = 8 and orderDetail.o_id = orders.o_id);
COMMIT;
SELECT * FROM customer;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

c_id	c_name	email	phone_number	balance	address
1	Alice Brown	alice@example.com	123-456-7890	78	123 Main St, Anytown, USA
2	Charlie Green	charlie@example.com	987-654-3210	200	456 Oak St, Othertown, USA
NULL	NULL	NULL	NULL	NULL	NULL

Figure 27: Customers' updated balances

2. Selecting the employee of the year and awarding them a raise

e_id	e_name	salary	hire_date	d_id
1	John Doe	60000.00	2020-01-15	3
2	Jane Smith	65000.00	2019-03-10	2
3	Bob Johnson	55000.00	2021-07-01	1
NULL	NULL	NULL	NULL	NULL

Figure 28: Salaries of employees

```
START TRANSACTION; -- Employee of the year
SET @best_employee := NULL;
SELECT e_id INTO @best_employee FROM performanceevaluation ORDER BY product_number DESC LIMIT 1;
UPDATE employee
SET SALARY = SALARY * 1.025
WHERE e_id = @best_employee;
COMMIT;
SELECT * FROM employee
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

e_id	e_name	salary	hire_date	d_id
1	John Doe	61500.00	2020-01-15	3
2	Jane Smith	65000.00	2019-03-10	2
3	Bob Johnson	55000.00	2021-07-01	1
NULL	NULL	NULL	NULL	NULL

Figure 29: Updated salaries of employees

3. Updating product price

p_id	p_name	price	stock_quantity	e_id
1	Product A	10.93	96	1
2	Product B	20.75	200	2
NULL	NULL	NULL	NULL	NULL

Figure 30: Product and their prices

```
157 •   START TRANSACTION;
158
159 •   UPDATE product SET price = price * 1.5 WHERE p_id = 2;
160
161 •   COMMIT;
162
163 •   SELECT * from product
```

Result Grid | Filter Rows: | Edit: | Export/Im

p_id	p_name	price	stock_quantity	e_id
1	Product A	10.93	96	1
2	Product B	31.13	200	2
NULL	NULL	NULL	NULL	NULL

Figure 31: Products' updated prices

12. Privileges and Roles

There are 3 roles that have been created in the database. The definition of these roles are as follows:

- **çağrı**: The administrator of the database so, this role has all the privileges.
- **employee**: Like the role of an administrator, but without the ability to list customers or link customers with products, ensuring customer privacy is maintained.
- **user**: User can only list the products and update order details.

user	host
admin	%
employee	%
user	%
çağrı	%
mysql.infoschema	localhost
mysql.session	localhost
mysql.sys	localhost
root	localhost

Figure 32: Users of the database

Grants for çağrı@%
GRANT USAGE ON *.* TO `çağrı`@`%`
GRANT ALL PRIVILEGES ON `databaseproject`....

Figure 33: Privileges for user 'çağrı'

Grants for user@%
GRANT USAGE ON *.* TO `user`@`%`
GRANT UPDATE ON `databaseproject`.`orderdetai...
GRANT SELECT ON `databaseproject`.`product` T...

Figure 34: Privileges for user 'user'

Grants for employee@%
GRANT USAGE ON *.* TO `employee`@`%`
GRANT SELECT, INSERT, UPDATE, DELETE ON `databaseproject`.`employe...
GRANT SELECT, INSERT, UPDATE, DELETE ON `databaseproject`.`orderde...
GRANT SELECT, INSERT, UPDATE, DELETE ON `databaseproject`.`orders` ...
GRANT SELECT, INSERT, UPDATE, DELETE ON `databaseproject`.`product...
GRANT SELECT, INSERT, UPDATE, DELETE ON `databaseproject`.`shipmen...

Figure 35: Privileges for user 'employee'

13. SQL Statements

All SQL commands used in this project are covered in the source codes of the database and the codes of the user interface.

14. REFERENCES

1. DBMS Normalization: 1NF, 2NF, 3NF and BCNF with Examples - javatpoint
2. Boyce-Codd Normal Form (BCNF) - GeeksforGeeks
3. draw.io (diagrams.net)
4. dbdiagram.io - Database Relationship Diagrams Design Tool